

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №1
по дисциплине «Вычислительная математика»

Вариант: 8

Преподаватель:
Зинченко Антон Андреевич

Выполнил: Некрутенко Максим Владимирович
Группа: P3206

Санкт-Петербург, 2026 г

Цель работы

Изучить численные методы решения систем линейных алгебраических уравнений и реализовать один из них средствами программирования.

Описание метода

Идеей метода Гаусса с выбором главного элемента является такая перестановка уравнений, чтобы на k -ом шаге исключения ведущим элементом a_{ii} оказывался наибольший по модулю элемент k -го столбца. Т.е. на очередном шаге k в уравнениях, начиная от k до последнего ($i=k, k+1, \dots, n$) в столбце k выбирают максимальный по модулю элемент и строки i и k меняются местами. Это выбор главного элемента «по столбцу».

Листинг программы

```
from pathlib import Path

import numpy as np

class GaussMethod:

    def __init__(self, a, b):

        self.a = [row[:] for row in a]
        self.b = b[:]
        self.n = len(b)
        self.triangular_A = None
        self.triangular_b = None
        self.count = 0

    def compare_with_numpy(self):
        A_np = np.array(self.a)
        b_np = np.array(self.b)

        try:
            x_numpy = np.linalg.solve(A_np, b_np)
            det_numpy = np.linalg.det(A_np)

            print(f"NumPy: {x_numpy}")
            print(f"NumPy: {det_numpy}")
        except np.linalg.LinAlgError:
            print("NumPy: Система вырождена или несовместна")

    def solve(self):
        if not self.forward_elimination():
            return None
        return self.backward_substitution()

    def forward_elimination(self):
        A = [row[:] for row in self.a]
        b = self.b[:]
        n = len(A)
        for k in range(0, n - 1):
            max_val = abs(A[k][k])
            max_row = k
            for i in range(k + 1, n):
                if abs(A[i][k]) > max_val:
                    max_val = abs(A[i][k])
```

```

        max_row = i

    if max_val < 1e-12:
        print("Система вырождена или несовместна. Метод Гаусса неприменим.")
        self.triangular_A = None
        return False

    if max_row != k:
        A[k], A[max_row] = A[max_row], A[k]
        b[k], b[max_row] = b[max_row], b[k]
        self.count += 1

    for i in range(k + 1, n):
        factor = A[i][k] / A[k][k]
        for j in range(k, n):
            A[i][j] -= factor * A[k][j]
            b[i] -= factor * b[k]
    if abs(A[n - 1][n - 1]) < 1e-12:
        print("Система вырождена. Последний диагональный элемент = 0.")
        self.triangular_A = None
        return False

    self.triangular_A = A
    self.triangular_b = b
    return True

def backward_substitution(self):
    n = self.n
    x = [0.0] * n
    A = self.triangular_A
    b = self.triangular_b
    for i in range(n - 1, -1, -1):
        s = b[i]
        for j in range(i + 1, n):
            s -= A[i][j] * x[j]
        x[i] = s / A[i][i]
    return x

def determinant(self):
    if self.triangular_A is None:
        if not self.forward_elimination():
            return 0.0
    det = 1.0
    for i in range(self.n):
        det *= self.triangular_A[i][i]
    return det * (-1) ** self.count

def compute_residual(self, x):
    r = []
    for i in range(self.n):
        s = sum(self.a[i][j] * x[j] for j in range(self.n))
        r.append(s - self.b[i])
    return r

def outputs_matrix(self):
    if self.triangular_A is None:
        print("Система вырождена")
        return
    print("\nТреугольная форма:")
    for i in range(self.n):
        row = " ".join(f"{self.triangular_A[i][j]:8.4f}" for j in range(self.n))
        print(f"[{row}] | {self.triangular_b[i]:8.4f}")

```

```

def read_from_console():
    while True:
        try:
            n = int(input("Введите размерность матрицы (0<n<=20): "))
            if int(n) > 20 or int(n) < 1:
                print("Некорректная размерность матрицы")

            else:
                break

        except ValueError:
            print("Размерность матрицы n должна быть целым числом")

    a = []
    print("Введите матрицу построчно {n*n}. В ней должны быть только числа!: ")
    for i in range(n):
        while True:
            try:
                row_str = input().strip()
                if not row_str:
                    print("Строка не должна быть пустой")
                    continue

                s = list(map(float, row_str.split()))
                if len(s) != n:
                    print("В строке должно быть n элементов")
                    continue

                a.append(s)
                break
            except ValueError:
                print("Некорректный элемент: все элементы должны быть числами (например, 2, -3.5).")
                print("Повторите ввод строки.")

    print("Введите вектор правых частей b: ")
    while True:
        try:
            row_str_b = input().strip()
            if not row_str_b:
                print("Строка не должна быть пустой")
                continue

            b = list(map(float, row_str_b.split()))
            if len(b) != n:
                print("Векторов правых частей b должно быть n!")
                continue

            break

        except ValueError:
            print("Вектора правых частей b должны быть числами")

    return a, b

def read_from_file(filename):
    with open(filename, "r", encoding="utf-8") as file:
        lines = [line.strip() for line in file if line.strip()]
        if not lines:
            raise ValueError("Файл пустой")

    try:

```

```

        n = int(lines[0])
    except ValueError:
        raise ValueError("Первая строка должна содержать целое число n")

    if not (1 <= n <= 20):
        raise ValueError("Размерность n должна быть в диапазоне от 1 до 20")

    if len(lines) != n + 2:
        raise ValueError(f"Ожидалось {n + 2} строк данных, получено {len(lines)}")

    a, b = [], []
    for i in range(1, n + 1):
        parts = lines[i].split()
        if len(parts) != n:
            raise ValueError(f"Строка {i}: ожидалось {n} чисел (n коэффициентов + правая часть)")
        try:
            row = list(map(float, parts))
        except ValueError:
            raise ValueError(f"Строка {i}: все элементы должны быть числами")

        a.append(row)

    parts_b = lines[n + 1].split()
    if len(parts_b) != n:
        raise ValueError(f"Последняя строка: ожидалось {n} чисел для вектора b")
    try:
        b = list(map(float, parts_b))
    except ValueError:
        raise ValueError("Вектор b: все элементы должны быть числами")

    return a, b

def file_exist(filename):
    return Path(filename).is_file()

def main():
    while True:
        print("Как вводить матрицу: 1-клавиатура, 2-из файла: ")
        s = input()
        if s == '1':
            a, b = read_from_console()
            break

        elif s == '2':
            while True:
                filename = input("Введите имя файла: ").strip()
                if not filename:
                    print("Имя файла не может быть пустым")
                    continue
                if not file_exist(filename):
                    print(f"Файл '{filename}' не найден. Попробуйте снова.")
                    continue
                break

            try:
                a, b = read_from_file(filename)
            except ValueError as e:
                print(f"Ошибка: {e}!")

```

```

else:
    print("Введите 1 или 2")

solver = GaussMethod(a, b)
x = solver.solve()
print("Решение:", x)
print("Определитель:", solver.determinant())
print("Невязка:", solver.compute_residual(x))
solver.outputs_matrix()

print("-----")
solver.compare_with_numpy()

if __name__ == '__main__':
    main()

```

Примеры работы программы

Как вводить матрицу: 1-клавиатура, 2-из файла:

2

Введите имя файла: matrix.txt

Решение: [2.375, 0.25, 1.875]

Определитель: 8.0

Невязка: [0.0, 0.0, 0.0]

Треугольная форма:

[2.0000 1.0000 0.0000] | 5.0000

[0.0000 2.5000 1.0000] | 2.5000

[0.0000 0.0000 1.6000] | 3.0000

NumPy: [2.375 0.25 1.875]

NumPy: 8.0000000000000002

Вывод:

В результате выполнения данной лабораторной работой я познакомился с численными методами решения математических задач на примере систем алгебраических уравнений, реализовав на языке программирования Python метод Гаусса с выбором главного элемента.